

Documento técnico basado en la implementación actual

Grenlus Signage

Ventajas técnicas, requisitos no funcionales y comportamiento de contenidos para una red de hasta 20 pantallas Android.

Alcance: backend Spring Boot, PostgreSQL, panel React/Vite y player Android.

Fecha: 19 de septiembre de 2026

Conclusión ejecutiva. El sistema está diseñado con sincronización dirigida por versión, reproducción offline y almacenamiento local en cada TV. Las consultas de control son livianas; el principal factor de carga son las descargas de archivos grandes cuando cambia una playlist.

1. Resumen técnico

Grenlus Signage separa el panel de administración, la API central, la base de datos y el reproductor Android. La oficina define el estado deseado de cada pantalla; la TV lo consulta y converge hacia ese estado cuando tiene conectividad.



Ventajas técnicas verificables

Continuidad operativa

La TV inicia con los contenidos que ya tiene en disco y puede seguir reproduciendo si el servidor o Internet no están disponibles.

Actualizaciones incrementales

La TV compara el par playlist/versión y solo descarga cuando detecta una actualización.

Integridad de descargas

Los archivos se descargan a un temporal y se renombran al finalizar; además se valida el tamaño esperado.

Aislamiento por pantalla

El player usa código y token; el backend verifica que el contenido esté en la playlist de esa pantalla.

Protección del almacenamiento

Se eliminan sobrantes y se comprueba el espacio disponible antes de descargar.

Observabilidad básica

El heartbeat periódico permite derivar el estado online/offline desde la última conexión.

Estas ventajas describen el código actual; no implican todavía CDN, almacenamiento de objetos ni escalado horizontal automático.

2. Flujo de archivos y contenidos

2.1 Alta desde el panel

1. El usuario selecciona una imagen o video en el panel.
2. El frontend envía un formulario multipart a la API.
3. El backend deduce si es imagen o video, genera un nombre UUID y guarda el archivo en el directorio de storage.
4. PostgreSQL conserva metadatos: nombre, tipo, tamaño, fecha, cliente y ruta interna.
5. La playlist incrementa su versión al modificarse, lo que permite a las TVs detectar el cambio.

Límite implementado: Spring permite hasta 500 MB por archivo y 500 MB por solicitud. El valor está en `application.properties` .

2.2 Consulta y descarga de una TV

1. Cada TV consulta `GET /api/player/{codigo}/config` .
2. Envía el token mediante el header `X-Pantalla-Token` .
3. La API responde con playlist, versión, estado de encendido y URLs de los contenidos.
4. Si la versión local coincide, no descarga nada.
5. Si cambió, descarga los faltantes desde `/api/player/{codigo}/contenidos/{id}/archivo` .

2.3 Seguridad de la entrega

El endpoint del player no exige el JWT del panel, porque la TV se autentica con su propio token. Antes de servir el archivo, el backend valida el código/token y verifica que el contenido pertenezca a la playlist de la pantalla.

3. Guardado local y operación offline

La TV guarda los archivos dentro del almacenamiento privado de la aplicación, en una carpeta de contenidos. El índice local contiene nombre, tipo y duración de cada archivo. También persiste la identidad de la pantalla y el par playlist/versión.

Situación	Comportamiento implementado
Se apaga la PC	La TV continúa con el contenido previamente descargado.
Se reinicia la TV sin Internet	Arranca usando la playlist local, si existe.
Se corta una descarga	Conserva un archivo parcial y puede pedir el tramo restante con <code>Range</code> , si el servidor responde adecuadamente.
La descarga queda incompleta	No cambia la playlist activa; conserva la anterior y reintenta en la siguiente sincronización.
Se quita contenido de una playlist	Después de aplicar la nueva playlist, elimina archivos sobrantes.
No hay espacio suficiente	Falla la actualización con un error explícito y la playlist anterior queda disponible.
La oficina apaga la pantalla	La TV puede quedar en negro, pero conserva los archivos para reanudar sin descargar todo.

Protección contra saturación del almacenamiento local

Antes de descargar, el player conserva lo que usa la playlist actual y lo que necesita la nueva, elimina archivos que no sirven y exige un margen de 200 MB. Esto evita acumular indefinidamente promociones antiguas.

Precaución: el límite real sigue siendo la capacidad libre del TV Box. Una playlist actual demasiado grande no puede instalarse; el sistema conserva la anterior, pero no puede crear espacio físico adicional.

4. Capacidad y saturación con 20 TVs

4.1 Tráfico de control

El player consulta configuración cada 30 segundos y envía heartbeat cada 20 segundos.

Para 20 TVs	Estimación derivada del intervalo actual
Consultas de configuración	40 por minuto; 2.400 por hora.

Heartbeats	60 por minuto; 3.600 por hora.
Transferencia de multimedia	Variable; ocurre principalmente al detectar una playlist nueva o un contenido faltante.

Las consultas y heartbeats son pequeños comparados con la transferencia de videos. No se puede afirmar una capacidad máxima sin medir el proveedor, CPU, memoria, ancho de banda, tamaño de archivos y concurrencia real.

4.2 Escenario de mayor carga

Si un video de 500 MB se asigna por primera vez a 20 TVs y todas descargan al mismo tiempo, la transferencia total puede aproximarse a:

$500 \text{ MB} \times 20 \text{ TVs} = \text{aproximadamente } 10 \text{ GB de salida.}$

Ese es el principal riesgo de saturación: ancho de banda, conexiones simultáneas, límites del hosting y costo de transferencia. El código reduce descargas repetidas, pero no distribuye todavía la carga mediante CDN ni escalona automáticamente las TVs.

Requisito no funcional de capacidad

Para operar 20 pantallas se debe dimensionar el hosting por el peor caso de actualización simultánea, no solo por las consultas periódicas.

5. Requisitos no funcionales

Área	Requisito / garantía actual	Precaución o pendiente
Disponibilidad	La TV reproduce la copia local ante una caída de conectividad.	El backend y el storage deben tener alta disponibilidad para nuevas altas y actualizaciones.
Rendimiento	La reproducción es local; no consume red por cada ciclo del video.	Las descargas iniciales o masivas requieren ancho de banda suficiente.
Escalabilidad	La detección por versión evita volver a descargar contenidos sin cambios.	No hay CDN, cola de descargas ni límite global de concurrencia implementado.
Persistencia	PostgreSQL persiste metadatos y cada TV persiste su caché local.	<code>./uploads</code> en un hosting efímero puede perderse; usar disco persistente u object storage.
Seguridad	JWT para panel; código/token para player; el player solo ve su playlist.	Usar HTTPS, secretos por variables de entorno y rotación de tokens.
Integridad	Descarga temporal, validación de tamaño y renombrado final.	Verificar en producción soporte real de HTTP Range, Content-Length y Content-Range.
Operabilidad	Heartbeat cada 20 s y estado basado en última conexión.	Agregar métricas, alertas, logs centralizados y monitoreo de almacenamiento.
Mantenibilidad	La sincronización está separada en player, API y servicio de contenidos.	Para producción, reemplazar <code>ddl-auto=update</code> por migraciones versionadas.

6. Precauciones para producción

1. **Almacenamiento:** mover imágenes y videos de `./uploads` a Cloudflare R2, Azure Blob o un disco persistente. PostgreSQL no contiene los binarios.
2. **HTTPS:** configurar una URL pública HTTPS para que las TVs no dependan de IPs locales ni de HTTP sin cifrado.
3. **Secretos:** reemplazar el secreto JWT de desarrollo y la contraseña inicial por variables privadas del proveedor.
4. **CORS:** permitir únicamente el dominio real del panel Vercel.
5. **Prueba de recuperación:** cortar Internet durante una descarga y verificar que la TV conserve la playlist anterior y retome correctamente.
6. **Prueba de carga:** simular 20 TVs descargando un archivo grande antes de contratar el plan definitivo.
7. **Actualizaciones simultáneas:** considerar una demora aleatoria o distribución por grupos si se publican videos grandes a muchas TVs.

8. **Capacidad local:** definir un tamaño máximo de playlist compatible con el almacenamiento real de cada modelo de TV Box.
9. **Limpieza del storage:** implementar una tarea para eliminar archivos huérfanos del servidor/object storage; la baja lógica actual conserva deliberadamente el archivo físico.

Recomendación: con 20 TVs, el diseño actual es técnicamente adecuado como base porque las TVs reproducen localmente y sincronizan por versión. La mejora prioritaria para una operación comercial es desacoplar los multimedia del disco local del backend y medir el peor caso de descargas simultáneas.

7. Evidencia técnica consultada

- `backend/src/main/resources/application.properties` : límites de subida, storage, CORS, JWT y PostgreSQL.
- `backend/.../ContenidoService.java` : escritura UUID en storage y metadatos en PostgreSQL.
- `backend/.../PlayerService.java` : configuración por playlist/versión y autorización por pantalla.
- `backend/.../PlayerController.java` : endpoints de configuración, descarga y heartbeat.
- `player-android/.../Sincronizador.kt` : caché local, limpieza, espacio libre y actualización atómica.
- `player-android/.../ApiPlayer.kt` : timeouts, token, descarga parcial y verificación de tamaño.
- `player-android/.../PlayerActivity.kt` : intervalos de 20/30 segundos y reproducción offline.
- `docker-compose.yml` : PostgreSQL local con volumen persistente y healthcheck.

Este documento no afirma características no implementadas. CDN, object storage, escalado horizontal, colas, métricas avanzadas y límites de concurrencia se presentan como precauciones o mejoras futuras, no como capacidades actuales.